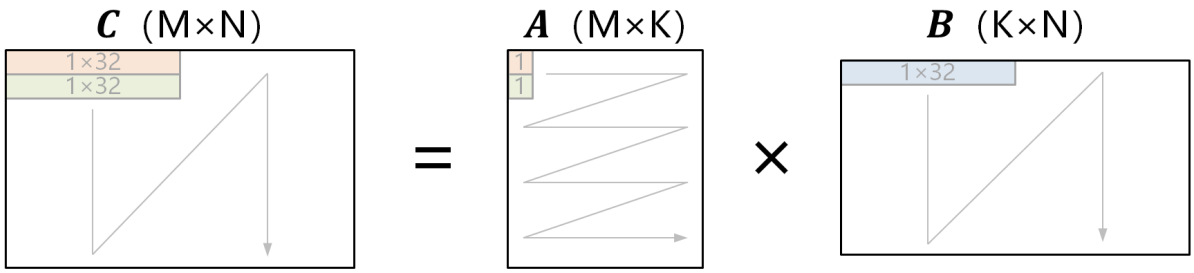
练习2：基于AVX指令设计并实现高性能矩阵乘法计算内核

- **任务**：利用AVX指令实现性能更好的矩阵乘法计算内核。
- **子任务**：
  - 计算内核 矩阵（）的元素个数不少于64，即；
  - 计算内核各维度必须满足如下要求: , , ;
  - 计算内核采用流水线设计读取、计算、写回等过程。
- **目标**：学会利用向量指令实现计算性能更高的矩阵乘法计算单元。

0. 模板代码 `src/lab3/gemm_kernel_opt_avx.S` 的实现思路

模板代码 `src/lab3/gemm_kernel_opt_avx.S` 采用如下图所示的矩阵分块算法：




```
for (n = 0; n < N; n += 32)
  for (m = 0; m < M; m += 2)
    for (k = 0; k < K; k++)
    {
```

$$\begin{bmatrix} C_{m,n} & C_{m,n+1} & \cdots\cdots & C_{m,n+31} \end{bmatrix} += a_{m,k} \times \begin{bmatrix} B_{k,n} & B_{k,n+1} & \cdots\cdots & B_{k,n+31} \end{bmatrix}$$

$$\begin{bmatrix} C_{m+1,n} & C_{m+1,n+1} & \cdots\cdots & C_{m+1,n+31} \end{bmatrix} += a_{m+1,k} \times \begin{bmatrix} B_{k,n} & B_{k,n+1} & \cdots\cdots & B_{k,n+31} \end{bmatrix}$$

```
    }
```

最内层循环每次使用 的 `A[m:m+2][k]` 分块和 的 `B[k][n:n+32]` 分块计算出 的 `C[m:m+2][n:n+32]` 分块。最内层的分块矩阵乘法采用 AVX 向量指令实现并行计算。

1. 为 `src/lab3/gemm_kernel_opt_avx.S` 的 `LOAD_MAT_C` 过程（第110行、第117行）添加加载 `C[m:m+2][n:n+32]` 到AVX寄存器的逻辑

Loading [Contrib]/a11y/accessibility-menu.js

```
110     // TODO 练习3: 请添加加载C[m][n:n+32]-->mat_c0_0_8, mat_c0_8_16, mat_c0_16_24, mat_c0_24_32的逻辑
111
112     // 装载矩阵C第二行的数据, 即C[m+1][n:n+32]
113     mov temp_reg, mat_elem_idx
114     add DIM_N, mat_elem_idx
115     shl $2, mat_elem_idx        // 左移, 相当于乘4
116
117     // TODO 练习3: 请添加加载C[m+1][n:n+32]-->mat_c1_0_8, mat_c1_8_16, mat_c1_16_24, mat_c1_24_32的逻辑
```

2. 为 `src/lab3/gemm_kernel_opt_avx.S` 的 `LOAD_MAT_A` 过程（第90行）添加加载 `A[m+1][k]` 到AVX寄存器的逻辑

```
88      vbroadcastss (MAT_A, mat_elem_idx), mat_a0_0_8    // 将A[m][k]广播到AVX寄存器的8个单元
89
90      // TODO 练习3: 请添加加载并广播A[m+1][k]-->mat_a1_0_8的逻辑
```

3. 为 `src/lab3/gemm_kernel_opt_avx.S` 的 `LOAD_MAT_B` 过程（第96行）添加加载 `B[k][n:n+32]` 到AVX寄存器的逻辑

```
94  .macro LOAD_MAT_B    // 每次装载矩阵B一行32个元素, 即B[k][n:n+32]
95
96      // TODO 练习3: 请添加加载B[k][n:n+32]-->mat_b0_0_8, mat_b0_8_16, mat_b0_16_24, mat_b0_24_32的逻辑
97
98  .endm
```

4. 为 `src/lab3/gemm_kernel_opt_avx.S` 的 `DO_COMPUTE` 过程（第141行）添加计算 `C[m:m+2][n:n+32]+=A[m:m+2][k]*B[k:k+8][n:n+32]` 的逻辑

```
139 .macro DO_COMPUTE       // 计算 C[m:m+2][n:n+32] += A[m:m+2][k] * B[k:k+8][n:n+32]
140
141     // TODO 练习3: 请添加计算C[m:m+2][n:n+32] += A[m:m+2][k] * B[k:k+8][n:n+32]的逻辑
142
143 .endm
```

5. 为 `src/lab3/gemm_kernel_opt_avx.S` 的 `STORE_MAT_C` 过程（第132行、第135行）添加保存 `C[m][n:n+32]` 和 `C[m+1][n:n+32]` 的逻辑

```
132     // TODO 练习3: 请添加保存mat_c0_0_8, mat_c0_8_16, mat_c0_16_24, mat_c0_24_32 --> C[m][n:n+32]的逻辑
133
134     // 保存矩阵C第二行的数据, 即C[m+1][n:n+32]
135     // TODO 练习3: 请添加保存mat_c1_0_8, mat_c1_8_16, mat_c1_16_24, mat_c1_24_32 --> C[m+1][n:n+32]的逻辑
```

6. 验证新内核的正确性

```
# build目录下执行
cmake -B . -S ../ && cmake --build ./ --target lab3_gemm_opt_avx.unittest
./dist/bins/lab3_gemm_opt_avx.unittest
```

Loading [Contrib]/a11y/accessibility-menu.js

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build$ ./dist/bins/lab3_gemm_
opt_avx.unittest
Running main() from /home/lenovo/test1/TA/try/computer_archtecture_labs/build/_d
eps/googletest-src/googletest/src/gtest_main.cc
[==========] Running 3 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 3 tests from gemm_kernel_opt_avx
[ RUN      ] gemm_kernel_opt_avx.test0
[       OK ] gemm_kernel_opt_avx.test0 (0 ms)
[ RUN      ] gemm_kernel_opt_avx.test1
[       OK ] gemm_kernel_opt_avx.test1 (0 ms)
[ RUN      ] gemm_kernel_opt_avx.test2
[       OK ] gemm_kernel_opt_avx.test2 (1 ms)
[----------] 3 tests from gemm_kernel_opt_avx (1 ms total)

[----------] Global test environment tear-down
[==========] 3 tests from 1 test suite ran. (1 ms total)
[  PASSED  ] 3 tests.
```

7. 对比优化后的算法与基线的性能（测试用例根据自己的优化选择），根据算法和结果分析优化方案的优缺点。

```
# build目录下执行
cmake -B . -S ../ && cmake --build ./ --target lab3_gemm_opt_avx
./dist/bins/lab3_gemm_opt_avx 256 256 256
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build$ ./dist/bins/lab3_gemm_
opt_avx 256 256 256
--- Performance before avx optimization ---
GEMM performance info:
                M, K, N: 256, 256, 256
                Ops: 0.0335544
                Total compute time(s): 3.55014
                Cost(s): 0.0177507
                Benchmark(Gflops): 1.89032
--- Performance for after avx optimization ---
GEMM performance info:
                M, K, N: 256, 256, 256
                Ops: 0.0335544
                Total compute time(s): 0.155425
                Cost(s): 0.000777125
                Benchmark(Gflops): 43.1777
----------------------------
Performance difference(Gflops): 41.2873
```

8. 测试并对比新内核在不同数据规模下的性能差异

改变输入矩阵的数据规模（比如边长为32、64、128等），测试AVX矩阵乘法内核的性能数据，绘制性能变化曲线图并进行合理分析。

Loading [Contrib]/a11y/accessibility-menu.js